

Trabalho N2 – Sistemas Operacionais

Alunos: Alessandra Pereira Bregadioli, Guilherme Souza da Silva Brai, Julia Unger

Monitorando Processos e Recursos do Sistema

Objetivo: Explorar gerenciamento de processos, CPU, memória e I/O.

Descrição:

Neste exercício, foi implementado um monitor simples de processos em C, utilizando acesso direto ao sistema de arquivos /proc do Linux.

O programa:

- Lista os processos ativos no sistema
- Lê estatísticas de CPU e memória diretamente do kernel (/proc/[PID]/stat e /proc/[PID]/status)
- Ordena os processos por uso de CPU e memória
- Exibe os 5 processos com maior consumo
- Exibe alertas quando:
 1. CPU > 50%
 2. Memória > 20%

Comandos utilizados:

Compilação: **gcc monitor.c -o monitor**

Execução: **./monitor**

Print da Execução:

```
brai@guilhermebrai-VMware: ~/Área de trabalho
brai@guilhermebrai-VMware:~/Área de trabalho$ gcc monitor.c -o monitor
brai@guilhermebrai-VMware:~/Área de trabalho$ ./monitor
===== TOP 5 CPU =====
PID=26089  USER=brai      CMD=monitor          CPU=  2.50% MEM=  0.04%
PID=1      USER=root      CMD=systemd          CPU=  0.00% MEM=  0.27%
PID=2      USER=root      CMD=kthreadd         CPU=  0.00% MEM=  0.00%
PID=3      USER=root      CMD=pool_workqueue_release CPU=  0.00% MEM=  0.00%
PID=4      USER=root      CMD=kworker/R-rcu_gp CPU=  0.00% MEM=  0.00%

===== TOP 5 MEM =====
PID=3402   USER=brai      CMD=firefox          MEM= 13.40% CPU=  0.00%
PID=4206   USER=brai      CMD=Isolated Web Co  MEM= 12.44% CPU=  0.00%
PID=2685   USER=brai      CMD=gnome-shell      MEM=  5.47% CPU=  0.00%
PID=5636   USER=brai      CMD=Isolated Web Co  MEM=  2.95% CPU=  0.00%
PID=5345   USER=brai      CMD=file:// Content  MEM=  2.71% CPU=  0.00%

===== ALERTAS (CPU>50.0% ou MEM>20.0%) =====
brai@guilhermebrai-VMware:~/Área de trabalho$
```

Comunicação entre Processos (IPC)

Objetivo: Entender como processos podem se comunicar:

- ipc_writer: envia mensagens PING N
- ipc_reader: recebe e responde PONG N <- PING N

Descrição

Foram criadas duas FIFOs no diretório de execução:

- fifo_in → canal de **writer** → **reader**
- fifo_out → canal de **reader** → **writer**

O reader abre leitura em fifo_in, lê a mensagem, e responde pela fifo_out.

O writer escreve em fifo_in e lê a resposta de fifo_out.

Comandos utilizados:

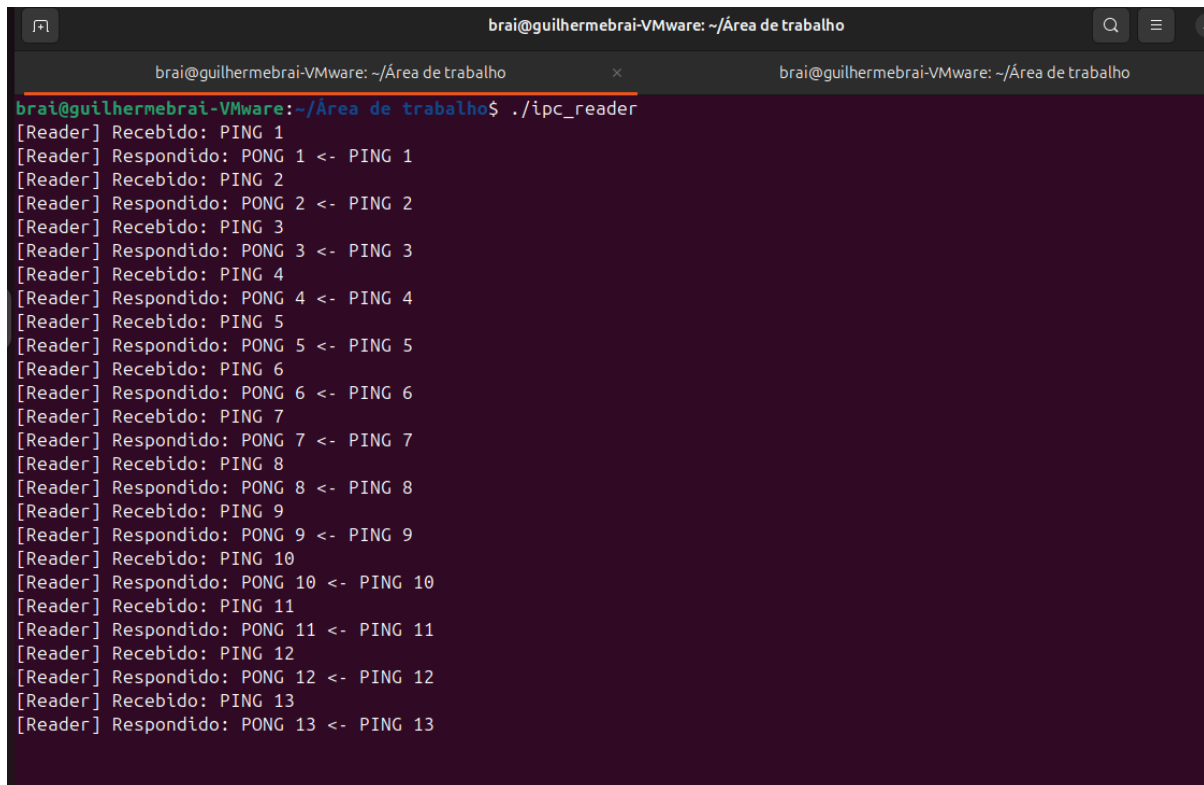
Compilação: **gcc -Wall -O2 ipc_reader.c -o ipc_reader** e **gcc -Wall -O2 ipc_writer.c -o ipc_writer**

Execução terminal A (READER): **./ipc_reader**

Execução terminal B (WRITER): `./ipc_writer`

Verificação das FIFOs: `ls -l fifo_in fifo_out`

Print da Execução:



```
brai@guilhermebrai-VMware: ~/Área de trabalho
brai@guilhermebrai-VMware: ~/Área de trabalho$ ./ipc_reader
[Reader] Recebido: PING 1
[Reader] Respondido: PONG 1 <- PING 1
[Reader] Recebido: PING 2
[Reader] Respondido: PONG 2 <- PING 2
[Reader] Recebido: PING 3
[Reader] Respondido: PONG 3 <- PING 3
[Reader] Recebido: PING 4
[Reader] Respondido: PONG 4 <- PING 4
[Reader] Recebido: PING 5
[Reader] Respondido: PONG 5 <- PING 5
[Reader] Recebido: PING 6
[Reader] Respondido: PONG 6 <- PING 6
[Reader] Recebido: PING 7
[Reader] Respondido: PONG 7 <- PING 7
[Reader] Recebido: PING 8
[Reader] Respondido: PONG 8 <- PING 8
[Reader] Recebido: PING 9
[Reader] Respondido: PONG 9 <- PING 9
[Reader] Recebido: PING 10
[Reader] Respondido: PONG 10 <- PING 10
[Reader] Recebido: PING 11
[Reader] Respondido: PONG 11 <- PING 11
[Reader] Recebido: PING 12
[Reader] Respondido: PONG 12 <- PING 12
[Reader] Recebido: PING 13
[Reader] Respondido: PONG 13 <- PING 13
```

```
brai@guilhermebrai-VMware: ~/Área de trabalho
brai@guilhermebrai-VMware: ~/Área de trabalho$ ./ipc_writer
[Writer] Enviado: PING 1
[Writer] Recebido: PONG 1 <- PING 1
[Writer] Enviado: PING 2
[Writer] Recebido: PONG 2 <- PING 2
[Writer] Enviado: PING 3
[Writer] Recebido: PONG 3 <- PING 3
[Writer] Enviado: PING 4
[Writer] Recebido: PONG 4 <- PING 4
[Writer] Enviado: PING 5
[Writer] Recebido: PONG 5 <- PING 5
[Writer] Enviado: PING 6
[Writer] Recebido: PONG 6 <- PING 6
[Writer] Enviado: PING 7
[Writer] Recebido: PONG 7 <- PING 7
[Writer] Enviado: PING 8
[Writer] Recebido: PONG 8 <- PING 8
[Writer] Enviado: PING 9
[Writer] Recebido: PONG 9 <- PING 9
[Writer] Enviado: PING 10
[Writer] Recebido: PONG 10 <- PING 10
[Writer] Enviado: PING 11
[Writer] Recebido: PONG 11 <- PING 11
[Writer] Enviado: PING 12
[Writer] Recebido: PONG 12 <- PING 12
[Writer] Enviado: PING 13
[Writer] Recebido: PONG 13 <- PING 13
[Writer] Enviado: PING 14
[Writer] Recebido: PONG 14 <- PING 14
[Writer] Enviado: PING 15

brai@guilhermebrai-VMware: ~/Área de trabalho$ ls -l fifo_in fifo_out
prw-rw-r-- 1 brai brai 0 nov  4 11:29 fifo_in
prw-rw-r-- 1 brai brai 0 nov  4 11:29 fifo_out
brai@guilhermebrai-VMware: ~/Área de trabalho$
```

Exercício de memória

Objetivo: Mostrar alocação de memória e consumo de processos.

Descrição:

Foi desenvolvido um programa que tenta alocar uma grande quantidade de memória dinamicamente.

Comandos utilizados:

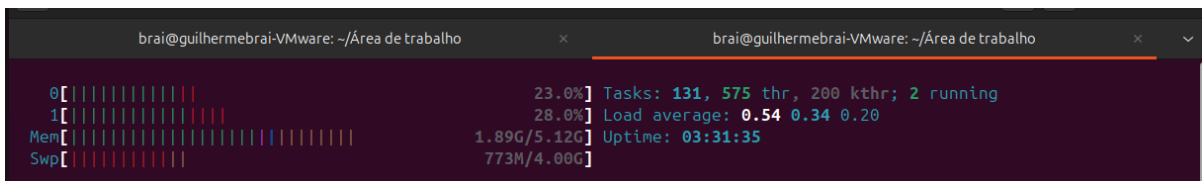
Compilação: **gcc memoria.c -o memoria**

Execução : **./memoria**

Monitoramento paralelo (outro terminal): **htop**

Print da Execução:

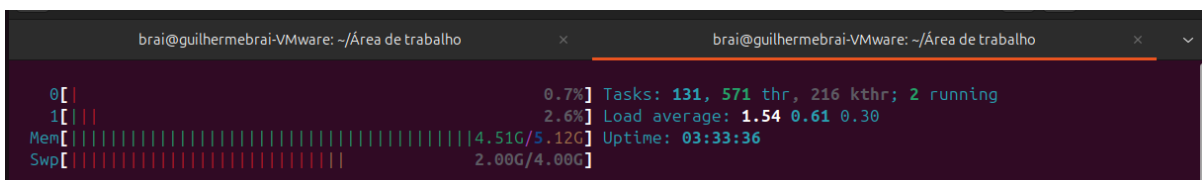
```
brai@guilhermebrai-VMware:~/Área de trabalho$ gcc memhog_safe.c -O2 -Wall -o memhog_safe
brai@guilhermebrai-VMware:~/Área de trabalho$ ./memhog_safe 256 4096
Iniciando: passo=256 MB, alvo=4096 MB
[OK] alocado: 256 MB / 4096 MB
[OK] alocado: 512 MB / 4096 MB
[OK] alocado: 768 MB / 4096 MB
[OK] alocado: 1024 MB / 4096 MB
[OK] alocado: 1280 MB / 4096 MB
[OK] alocado: 1536 MB / 4096 MB
[OK] alocado: 1792 MB / 4096 MB
[OK] alocado: 2048 MB / 4096 MB
[OK] alocado: 2304 MB / 4096 MB
[OK] alocado: 2560 MB / 4096 MB
[OK] alocado: 2816 MB / 4096 MB
[OK] alocado: 3072 MB / 4096 MB
[OK] alocado: 3328 MB / 4096 MB
[OK] alocado: 3584 MB / 4096 MB
[OK] alocado: 3840 MB / 4096 MB
[OK] alocado: 4096 MB / 4096 MB
Pausa: pressione ENTER para liberar memória (ou Ctrl+C pra manter).
```



A screenshot of a terminal window showing system status before running the script. The window title is 'brai@guilhermebrai-VMware: ~/Área de trabalho'. The terminal displays a progress bar with 23.0% completion, 28.0% load average, and 1.89G/5.12G memory usage. The system is running 131 tasks, 575 threads, and 200 kthrs. The uptime is 03:31:35.

Metric	Value
Progress	23.0%
Load Average	28.0%
Memory	1.89G/5.12G
Swap	773M/4.00G
Tasks	131, 575 thr, 200 kthrs
Running	2
Uptime	03:31:35

Memória antes do script.



A screenshot of a terminal window showing system status after running the script. The window title is 'brai@guilhermebrai-VMware: ~/Área de trabalho'. The terminal displays a progress bar with 0.7% completion, 2.6% load average, and 4.51G/5.12G memory usage. The system is running 131 tasks, 571 threads, and 216 kthrs. The uptime is 03:33:36.

Metric	Value
Progress	0.7%
Load Average	2.6%
Memory	4.51G/5.12G
Swap	2.00G/4.00G
Tasks	131, 571 thr, 216 kthrs
Running	2
Uptime	03:33:36

Memória depois do script.

Gerenciamento de arquivos

Objetivo: Trabalhar com estrutura de diretórios, permissões e operações de arquivos.

Descrição:

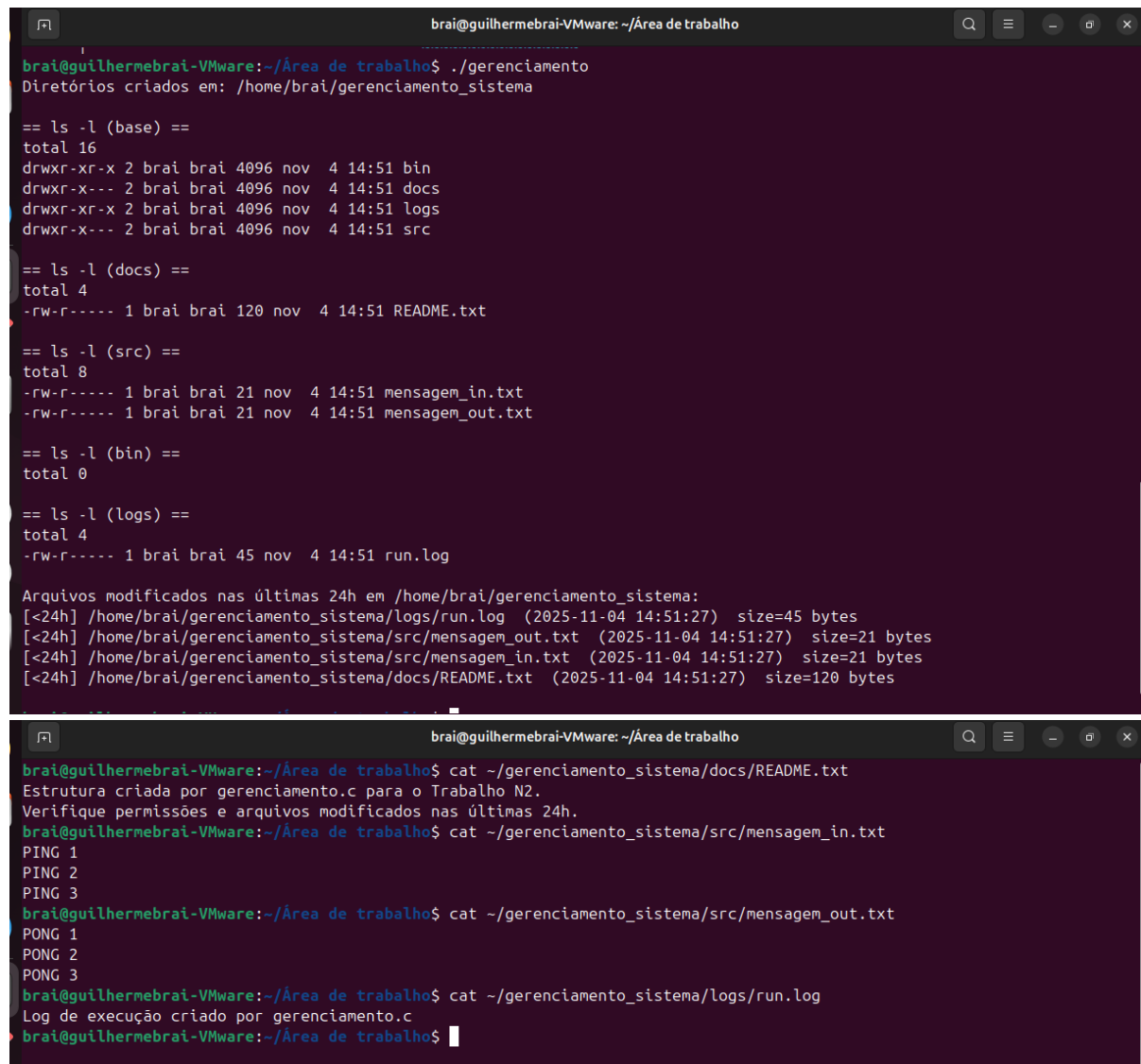
Neste exercício, o programa implementa um sistema de gerenciamento de arquivos em C, criando a estrutura de diretórios e arquivos, configurando permissões adequadas e listando os arquivos modificados recentemente.

Comandos utilizados:

Compilação: `gcc -std=c11 -O2 -Wall fsops.c -o fsops`

Execução: `./fsops`

Print da Execução:



```
brai@guilhermebrai-VMware: ~/Área de trabalho
brai@guilhermebrai-VMware:~/Área de trabalho$ ./gerenciamento
Diretórios criados em: /home/brai/gerenciamento_sistema

== ls -l (base) ==
total 16
drwxr-xr-x 2 brai brai 4096 nov  4 14:51 bin
drwxr-x-- 2 brai brai 4096 nov  4 14:51 docs
drwxr-xr-x 2 brai brai 4096 nov  4 14:51 logs
drwxr-x-- 2 brai brai 4096 nov  4 14:51 src

== ls -l (docs) ==
total 4
-rw-r----- 1 brai brai 120 nov  4 14:51 README.txt

== ls -l (src) ==
total 8
-rw-r----- 1 brai brai 21 nov  4 14:51 mensagem_in.txt
-rw-r----- 1 brai brai 21 nov  4 14:51 mensagem_out.txt

== ls -l (bin) ==
total 0

== ls -l (logs) ==
total 4
-rw-r----- 1 brai brai 45 nov  4 14:51 run.log

Arquivos modificados nas últimas 24h em /home/brai/gerenciamento_sistema:
[<24h] /home/brai/gerenciamento_sistema/logs/run.log (2025-11-04 14:51:27) size=45 bytes
[<24h] /home/brai/gerenciamento_sistema/src/mensagem_out.txt (2025-11-04 14:51:27) size=21 bytes
[<24h] /home/brai/gerenciamento_sistema/src/mensagem_in.txt (2025-11-04 14:51:27) size=21 bytes
[<24h] /home/brai/gerenciamento_sistema/docs/README.txt (2025-11-04 14:51:27) size=120 bytes

brai@guilhermebrai-VMware:~/Área de trabalho$ cat ~/gerenciamento_sistema/docs/README.txt
Estrutura criada por gerenciamento.c para o Trabalho N2.
Verifique permissões e arquivos modificados nas últimas 24h.
brai@guilhermebrai-VMware:~/Área de trabalho$ cat ~/gerenciamento_sistema/src/mensagem_in.txt
PING 1
PING 2
PING 3
brai@guilhermebrai-VMware:~/Área de trabalho$ cat ~/gerenciamento_sistema/src/mensagem_out.txt
PONG 1
PONG 2
PONG 3
brai@guilhermebrai-VMware:~/Área de trabalho$ cat ~/gerenciamento_sistema/logs/run.log
Log de execução criado por gerenciamento.c
brai@guilhermebrai-VMware:~/Área de trabalho$
```